

课程笔记

CS146S Week 2: The Anatomy of Coding Agents (MCP)

基于 Mihail Eric 授课内容整理

2025 年 10 月

课程: Stanford CS146S

学期: Fall 2025

讲次: 2 lectures (Mon + Fri)

网站: <https://themodernsoftware.dev>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 从零构建 Coding Agent	2
1.1 什么是 Coding Agent	2
1.2 “秘密武器”: Claude Code 的设计哲学	2
1.3 本章小结	2
2 Model Context Protocol (MCP)	3
2.1 为什么需要 MCP	3
2.2 MCP 是什么	3
2.3 MCP 架构详解	3
2.3.1 核心术语	3
2.3.2 工作流程	3
2.3.3 传输层	4
2.4 MCP 的局限性	4
2.5 本章小结	4
3 总结与延伸	4
3.1 关键点	4
3.2 拓展阅读	4

1 从零构建 Coding Agent

1.1 什么是 Coding Agent

Coding Agent 是一种能够自主执行编程任务的 AI 系统。其核心组件包括：

- **System Prompt**: 定义 Agent 的行为和指令
- **User Prompt**: 用户的自定义请求
- **Assistant Prompt**: LLM 的响应
- **Tool Set**: Agent 可调用的工具集合

Coding Agent 的核心循环

一个 Coding Agent 的基本工作流程极其简单：

1. 从终端读取用户输入，持续追加到对话历史
2. 告知 LLM 可用的工具（Read_file、List_dir、Edit_file 等）
3. LLM 在适当时机请求调用工具
4. 离线执行工具调用并返回结果
5. 重复上述过程直到任务完成

1.2 “秘密武器”：Claude Code 的设计哲学

深入分析 Claude Code 的底层设计，可以发现几个关键的工程决策：

1. **前置 context 注入**：使用小而精准的 prompt 进行 context 前置加载
2. **System reminders 无处不在**：在 system prompt、user prompt、tool calls、tool results 中都插入 <system-reminder> 标签，防止模型在长对话中偏离目标
3. **Command prefix extraction**：从用户输入中提取命令前缀，实现快捷操作
4. **Subagent 派生**：生成子 Agent 处理特定子任务，避免单个 context 过载

Context Drift 问题

在长对话中，LLM 容易“忘记”早期的指令和约束——这被称为 context drift。Claude Code 通过在对话的各个环节反复插入 system reminders 来对抗这一问题。这是一个经过工程验证的关键技巧。

1.3 本章小结

构建一个基础 Coding Agent 的技术门槛其实很低——核心就是 LLM + tool calling 的循环。但要构建一个可靠的 Coding Agent，需要大量的工程细节：context 管理、drift 防护、子任务分解等。

2 Model Context Protocol (MCP)

2.1 为什么需要 MCP

LLM 拥有广泛但**静态**的世界知识——只有在重训时才会更新。要构建真正自主的系统，需要可靠的方式向 LLM 注入动态数据（天气、股价、最新 API 文档等）。

RAG 和 tool calling 是目前最好的答案。但问题在于：

M × N 连接器问题

假设有 M 个 LLM 应用和 N 个第三方工具/API。在 MCP 之前，每个应用需要为每个工具编写独立的集成代码——包括认证、错误处理、速率限制等。这导致了 $M \times N$ 个连接器的维护负担。

2.2 MCP 是什么

MCP (Model Context Protocol) 是 Anthropic 于 2024 年 11 月推出的开放协议，为 LLM 提供了一种**标准化**的工具暴露格式。

MCP 的核心价值

MCP 将工具集成从 $M \times N$ 降低到 $M + N$ ：

- 不需要为每个工具重新实现认证、错误处理、速率限制
- 使用 JSON-RPC 强制执行一致的输出格式
- 从 Language Server Protocol (LSP) 扩展而来，但支持**主动的** (proactive) agentic workflow，而非 LSP 的纯反应式模式

2.3 MCP 架构详解

2.3.1 核心术语

- **Host**：宿主应用（如 Cursor、Claude Desktop）
- **MCP Client**：嵌入在 Host 中的库（为每个 Server 维护有状态的 session）
- **MCP Server**：轻量级的工具包装器 (wrapper)
- **Tool**：可调用的函数（数据源、API 等）

2.3.2 工作流程

1. Client 调用 `tools/list` 向 MCP Server 查询：“你能做什么？”
2. Server 返回 JSON 描述每个工具（名称、摘要、JSON Schema）
3. Host 将这些 JSON 注入模型的 context
4. 用户 prompt 触发模型，模型输出结构化的 tool call
5. MCP Server 执行调用，对话继续

2.3.3 传输层

MCP 提供两种传输机制：

- **stdio**：标准输入/输出，适用于本地进程通信
- **SSE** (Server-Sent Events)：适用于远程服务通信

2.4 MCP 的局限性

当前 Agent 处理多工具的能力有限

- Agent 在面对大量工具时表现不佳——工具描述会快速消耗 context window
- API 设计需要面向 AI 原生 (AI-native)，而非沿用传统的刚性 API 设计
- 复杂的 API 参数和嵌套结构会显著降低工具调用的准确率

2.5 本章小结

MCP 是 AI 编程工具生态的重要基础设施，通过标准化的协议解决了 LLM 与外部工具集成的 $M \times N$ 问题。理解 MCP 的架构 (Host $\rightarrow$ Client $\rightarrow$ Server $\rightarrow$ Tool) 和通信流程是构建和使用 AI 编程工具的关键。

3 总结与延伸

Week 2 深入了 Coding Agent 的技术内核。从手动构建一个最小化的 Coding Agent，到理解 MCP 协议如何将工具集成标准化，课程展示了 AI 编程工具“魔法”背后的工程实现。

3.1 关键点

- Coding Agent = LLM + Tool Calling 循环，核心简单但工程细节决定质量
- Claude Code 的关键设计：context 前置加载、system reminders 防 drift、subagent 分流
- MCP 将 $M \times N$ 集成降为 $M + N$ ，使用 JSON-RPC 标准化工具描述
- MCP 架构：Host $\rightarrow$ Client $\rightarrow$ Server $\rightarrow$ Tool
- 当前局限：Agent 处理多工具能力有限，API 需要 AI-native 设计

3.2 拓展阅读

- MCP 官方文档： <https://modelcontextprotocol.io>
- Anthropic MCP 发布博客： <https://www.anthropic.com/news/model-context-protocol>
- JSON-RPC 2.0 规范： <https://www.jsonrpc.org/specification>
- Language Server Protocol： <https://microsoft.github.io/language-server-protocol/>